Q1 CO4 - AT2 - CASE STUDY

25 marks

A large enterprise resource planning application is built from multiple source files, including modules for sales, inventory, and finance, and all of these modules must reference a common variable that stores the current fiscal year. The variable is defined once in one source file but must be read and, in some cases, modified by functions compiled separately in other source files. Without proper declaration, each module would either fail to compile or maintain its own inconsistent copy of the fiscal year. The design must ensure a single, consistent value is shared across the entire application.

Question: Identify which storage class allows the fiscal-year variable to be accessed across multiple source files, and explain the role of extern in achieving this shared access.

Your Code

```c
#include<stdio.h>
extern int fiscal_year;
extern void update_year(int new_year);
int main(){
   printf("current fiscal year: %d\n", fiscal_year);
    update_year(2025);
    printf("update fiscal year:%d\n",fiscal_year);
    return 0;
}
int fiscal_year =2024;
void update_year(int new_year){
    fiscal_year = new_year;
}
```

Input

stdin input

Output

Q2 CO4 - AT2 - CASE STUDY

25 marks

A smart utility meter installed in a residential complex can measure electricity consumption, water flow, or gas usage, but the hardware reports only one utility type per polling cycle depending on which sensor is currently active. Common information such as meter ID, timestamp, and location must always be recorded alongside whichever utility reading is captured. The billing server receives thousands of polling records daily and must interpret the correct reading type each time. Minimizing per-record memory footprint is a key design goal for the embedded firmware.

Question: Design a structure that embeds a union for the utility-specific reading, and explain why this hybrid representation is more memory-efficient than using three separate full-sized fields.

Your Code

```c
#include <stdio.h>
#include <string.h>
typedef enum {
    ELECTRICITY,
    WATER,
    GAS
} UtilityType;
typedef struct {
    int meterID;
    long timestamp;
    char location[30];
    UtilityType type;
    union {
        float electricity_kWh;
        float water_liters;
        float gas_m3;
    } reading;
} UtilityRecord;
```

Input

stdin input

Output

```
Record 1: ID 101, Loc: Block A, Apt 4, Type: 0, Value: 12.50 kWh
Record 2: ID 202, Loc: Block B, Apt 12, Type: 1, Value: 45.80 L
```

Q3 CO4 - AT2 - CASE STUDY

25 marks

An insurance company processes claims for health, vehicle, and property policies, where each claim type requires distinct supporting information such as hospital details, accident details, or property-damage details respectively. Every claim, regardless of type, shares common fields such as claim ID, policyholder name, and claim date. Claims officers repeatedly retrieve and update a claim's status as it moves through approval stages. The company wants a design that avoids wasting memory on fields that do not apply to a given claim type.
Question: Design a structure containing a union to represent claim-type-specific information, and explain how a structure pointer can be used to update a claim's status efficiently as it progresses through the workflow.

Your Code

```c
#include <stdio.h>
#include <string.h>
union ClaimDetails {
    char hospitalDetails[50];
    char accidentDetails[50];
    char propertyDamage[50];
};
struct Claim {
    int claimID;
    char policyholderName[50];
    char claimDate[15];
    char status[20];
    int claimType; //
    union ClaimDetails details;
};
void updateClaimStatus(struct Claim *c, const char *newStatus) {
    strcpy(c->status, newStatus);
    printf("Claim ID %d status updated to: %s\n", c->claimID, c->status
}

```

Input

```
stdin input
```

Output

```
--- Initial Claim Information ---
ID: 101
Holder: Jayanth Reddy
Status: Pending
Type: Health | Details: City
General Hospital
```

Q4 CO4 - AT2 - CASE STUDY

25 marks

A distribution warehouse maintains inventory records for tens of thousands of products, where each record stores a product code, product name, quantity in stock, and supplier address consisting of several related sub-fields. Warehouse staff frequently search for a specific product to check stock levels and update the quantity after every incoming or outgoing shipment. Because the inventory is large, copying entire records during routine updates would be inefficient. The system must remain responsive even during high-volume shipment processing.
Question: Design an array of structures to hold the inventory, and explain how a structure pointer allows staff to update a selected product's stock quantity without copying the complete record.

Your Code

```c
#include <stdio.h>
#include <string.h>
struct Address {
    char street[50];
    char city[30];
    char state[20];
};
struct Product {
    int productCode;
    char productName[50];
    int quantity;
    struct Address supplierAddress;
};
void updateStock(struct Product *ptr, int newQuantity) {
    ptr->quantity = newQuantity;
    printf("\nStock updated successfully for Product Code: %d\n", ptr->
}

```

Input

```
stdin input
```

Output

```
--- Warehouse Inventory System ---
Enter Product Code to update stock:
Product not found in inventory.
```